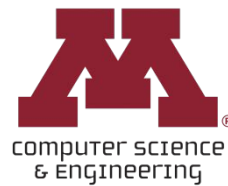


CSCI 5451: Introduction to Parallel Computing

Lecture 26: Reduction in Cuda



UNIVERSITY OF MINNESOTA
Driven to Discover®

Reduction (Recall)

Reductions involve an operation performed over some set of data



Reduction (Recall)

Reductions involve an operation performed over some set of data

Addition

```
01    sum = 0.0f;  
02    for(i = 0; i < N; ++i) {  
03        sum += input[i];  
04    }
```



Reduction (Recall)

Reductions involve an operation performed over some set of data

Addition

```
01    sum = 0.0f;  
02    for(i = 0; i < N; ++i) {  
03        sum += input[i];  
04    }
```

Generic Operations

```
01    acc = IDENTITY;  
02    for(i = 0; i < N; ++i) {  
03        acc = Operator(acc, input[i]);  
04    }
```



Reduction (Recall)

How to Parallelize?

Reductions involve an operation performed over some set of data

Addition

```
01    sum = 0.0f;
02    for(i = 0; i < N; ++i) {
03        sum += input[i];
04    }
```

Generic Operations

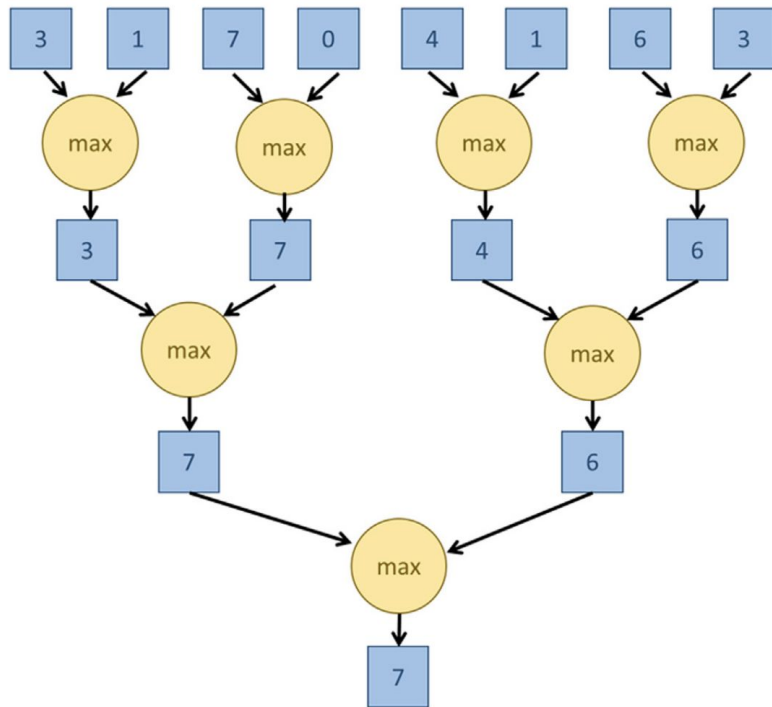
```
01    acc = IDENTITY;
02    for(i = 0; i < N; ++i) {
03        acc = Operator(acc, input[i]);
04    }
```



Reduction (Recall)

How to Parallelize?

Reduction Trees



Reductions involve an operation performed over some set of data

Addition

```
01    sum = 0.0f;
02    for(i = 0; i < N; ++i) {
03        sum += input[i];
04    }
```

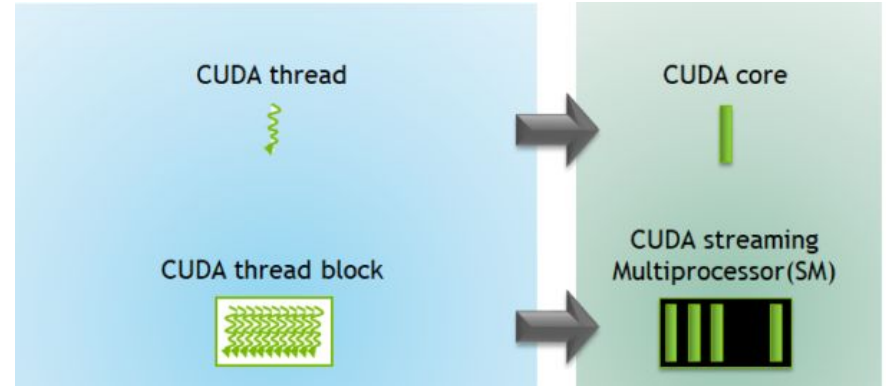
Generic Operations

```
01    acc = IDENTITY;
02    for(i = 0; i < N; ++i) {
03        acc = Operator(acc, input[i]);
04    }
```



Simple Kernel Within one Block

- ❑ To start with, let's write a reduction kernel using only a single threadblock
- ❑ If we have an array of N elements, what is the maximum amount of concurrency when performing a reduction
 -

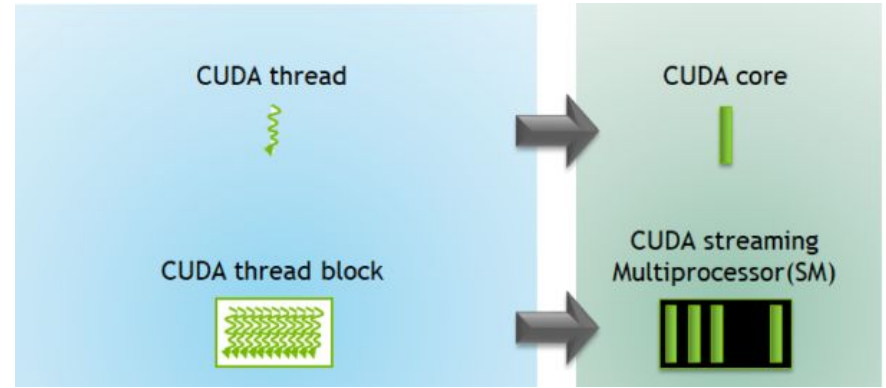


```
my_kernel<<<1, num_threads>>>(arg1, arg2);
```



Simple Kernel Within one Block

- ❑ To start with, let's write a reduction kernel using only a single threadblock
- ❑ If we have an array of N elements, what is the maximum amount of concurrency when performing a reduction
 - $N/2$
- ❑ Given this, what is the maximum array size we can perform reduction on in a single threadblock
 -

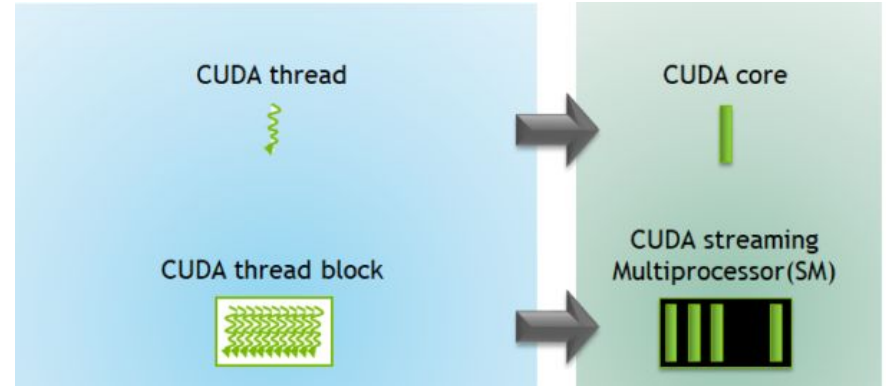


```
my_kernel<<<1, num_threads>>>(arg1, arg2);
```



Simple Kernel Within one Block

- ❑ To start with, let's write a reduction kernel using only a single threadblock
- ❑ If we have an array of N elements, what is the maximum amount of concurrency when performing a reduction
 - $N/2$
- ❑ Given this, what is the maximum array size we can perform reduction on in a single threadblock
 - 2048 (i.e. number of threads in a block times 2)



```
my_kernel<<<1, num_threads>>>(arg1, arg2);
```



Simple Kernel Within one Block

```
01  __global__ void SimpleSumReductionKernel(float* input, float* output) {
02      unsigned int i = 2*threadIdx.x;
03      for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
04          if (threadIdx.x % stride == 0) {
05              input[i] += input[i + stride];
06          }
07          __syncthreads();
08      }
09      if(threadIdx.x == 0) {
10          *output = input[0];
11      }
12  }
```



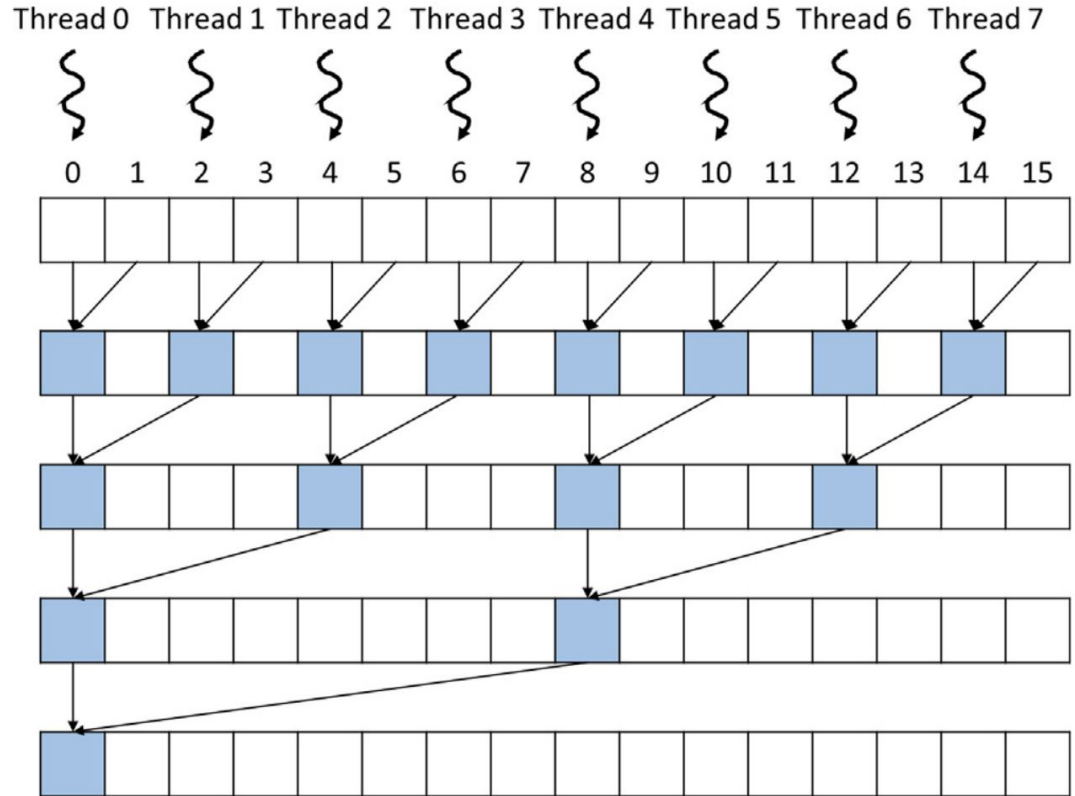
Simple Kernel Within one Block

What does this reduction tree look like?
Which threads access what elements of
memory? Which memory locations are
written to at each depth of the tree?

```
01  __global__ void SimpleSumReductionKernel(float* input, float* output) {
02      unsigned int i = 2*threadIdx.x;
03      for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
04          if (threadIdx.x % stride == 0) {
05              input[i] += input[i + stride];
06          }
07          __syncthreads();
08      }
09      if(threadIdx.x == 0) {
10          *output = input[0];
11      }
12  }
```



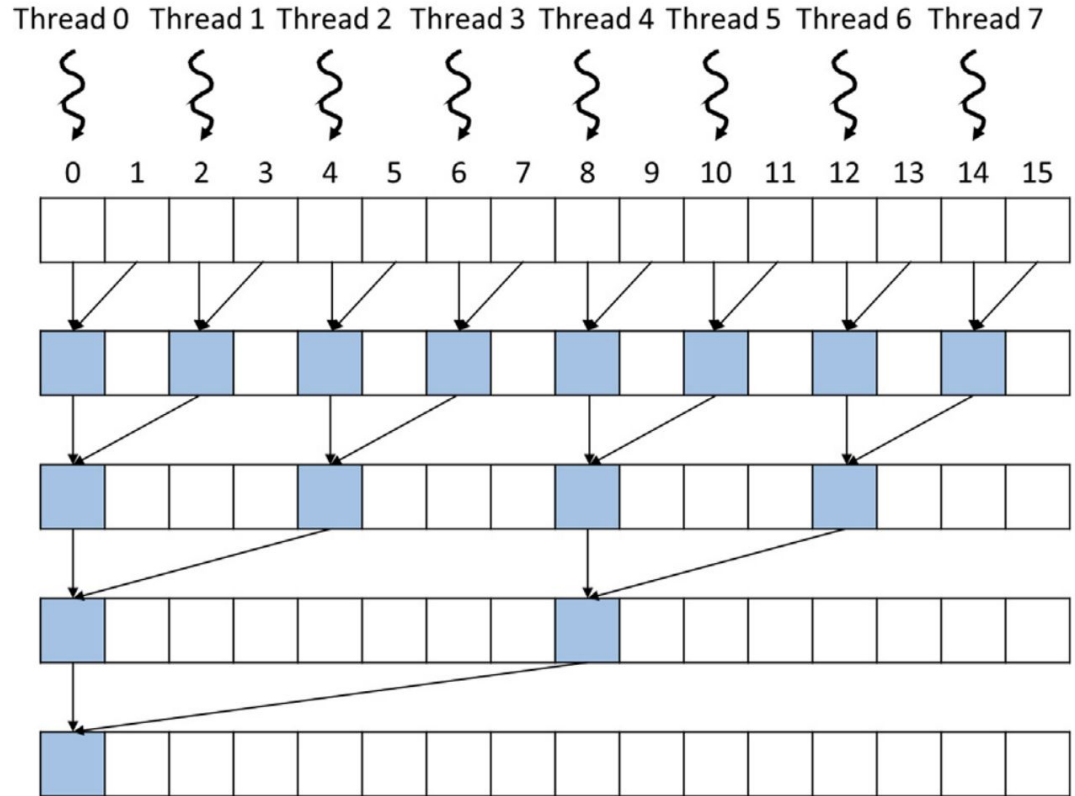
Simple Kernel Within one Block



Simple Kernel Within one Block

What are some problems with this implementation?

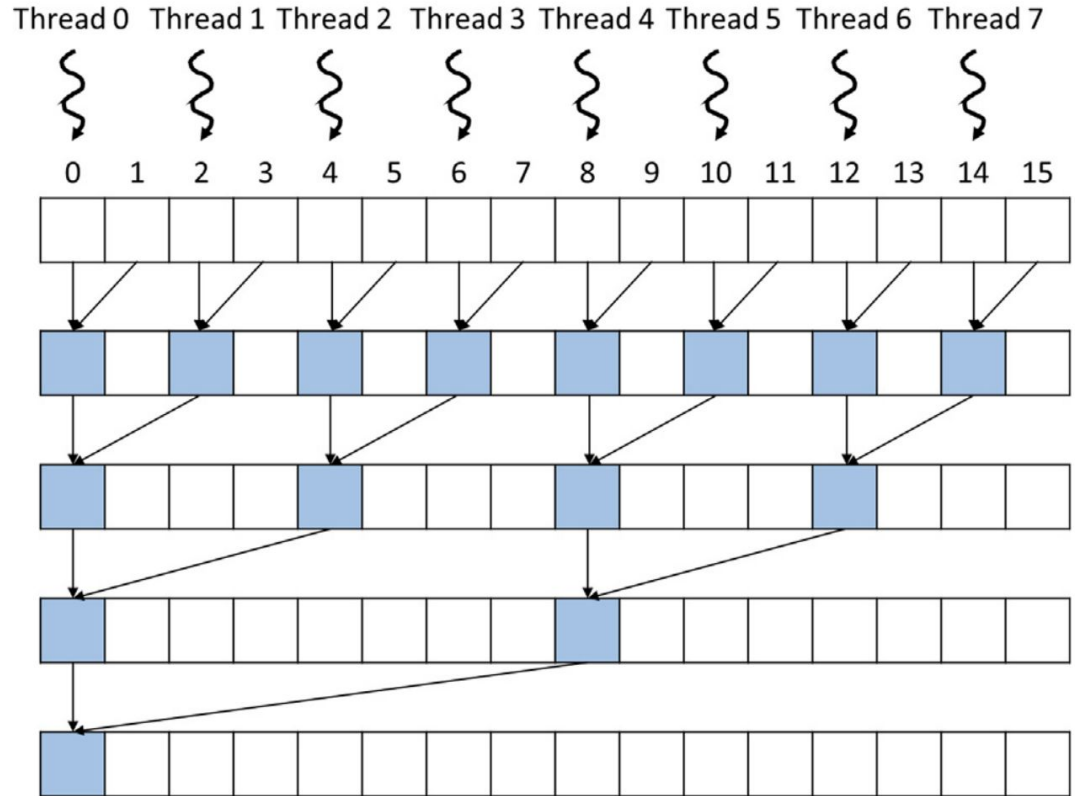
-



Simple Kernel Within one Block

What are some problems with this implementation?

- Warp Divergence.
- Divergent memory accesses (no coalescing)
- Redundant memory loading

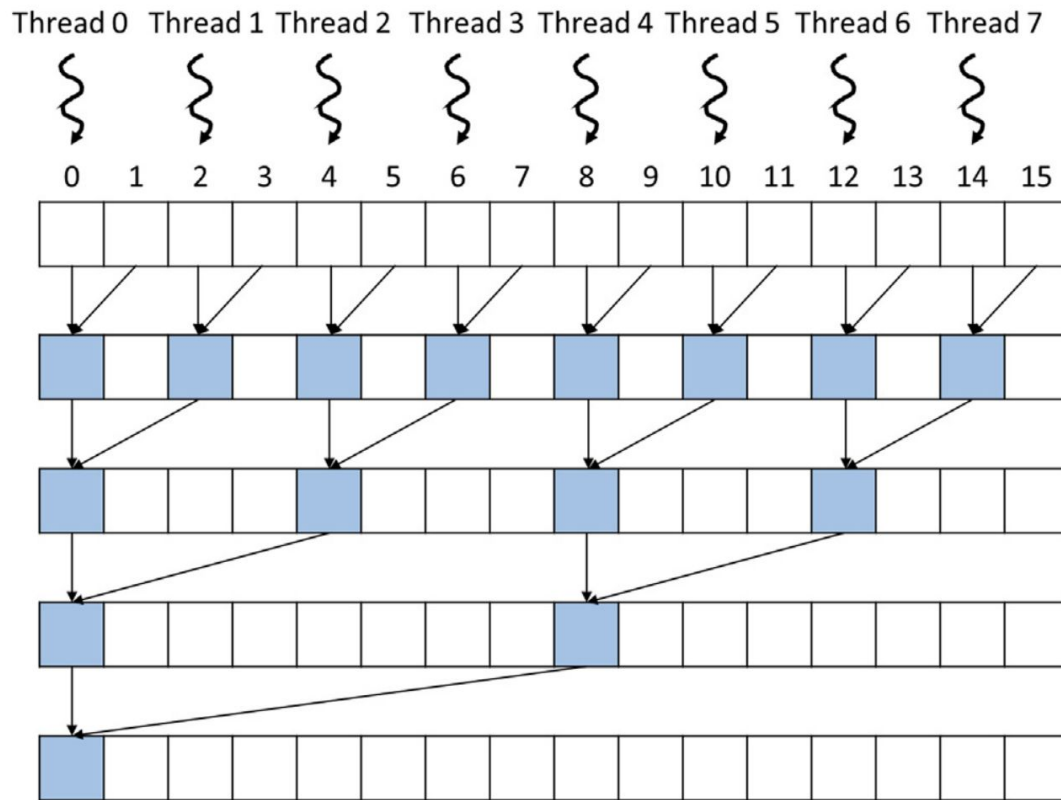


Simple Kernel Within one Block

What are some problems with this implementation?

- **Warp Divergence.**
- **Divergent memory accesses (no coalescing)**
- Redundant memory loading

Let's fix these first



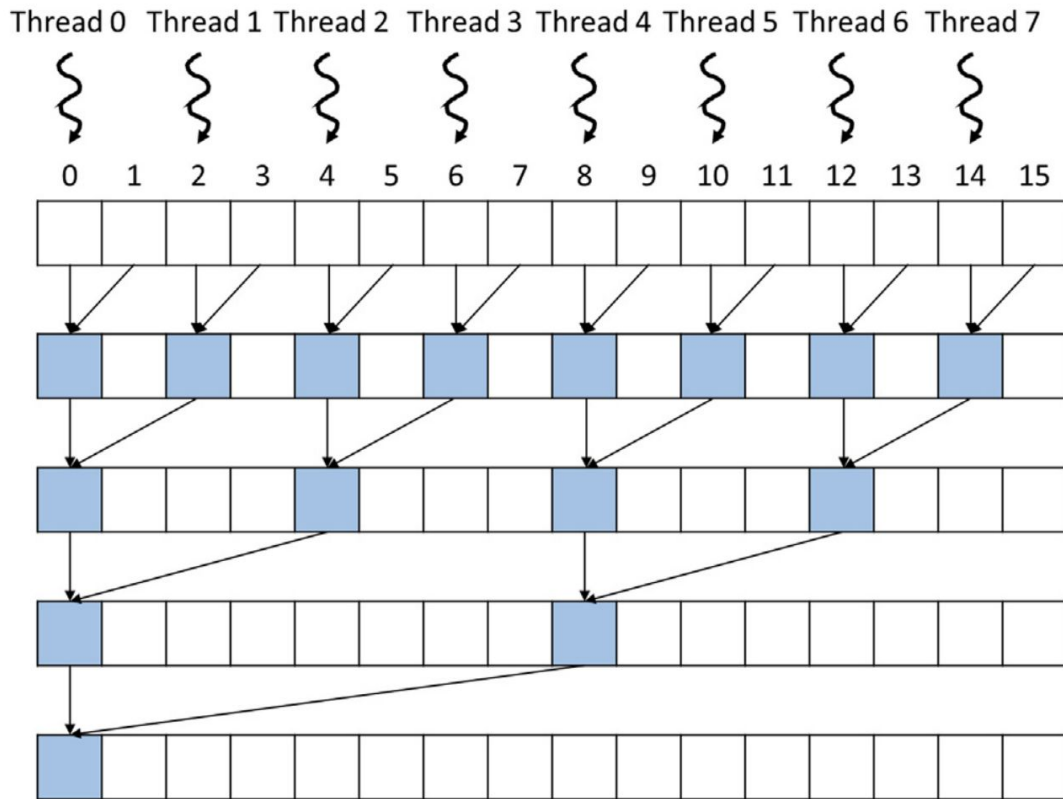
Simple Kernel Within one Block

How to fix this?

What are some problems with this implementation?

- **Warp Divergence.**
- **Divergent memory accesses (no coalescing)**
- Redundant memory loading

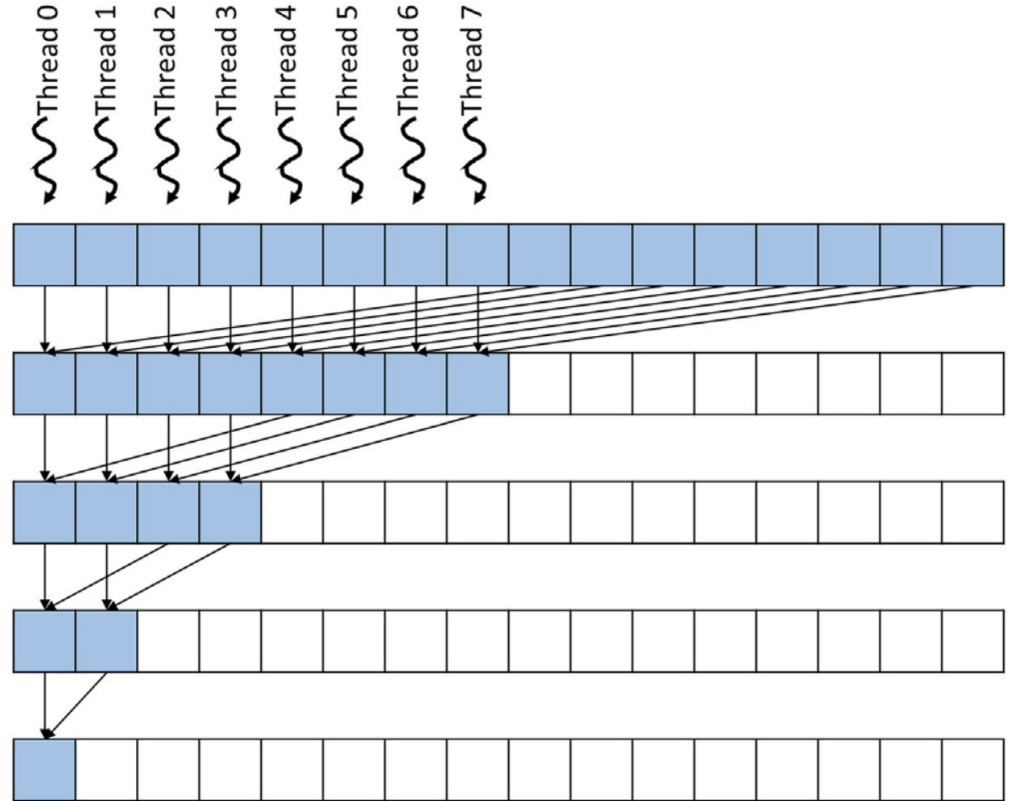
Let's fix these first



Minimize Divergence Kernel

Threads now compute over adjacent memory locations.

- No warp divergence
- Coalesced reads/writes



Minimize Divergence Kernel

```
01  __global__ void ConvergentSumReductionKernel(float* input, float* output) {
02      unsigned int i = threadIdx.x;
03      for (unsigned int stride = blockDim.x; stride >= 1; stride /= 2) {
04          if (threadIdx.x < stride) {
05              input[i] += input[i + stride];
06          }
07          __syncthreads();
08      }
09      if(threadIdx.x == 0) {
10          *output = input[0];
11      }
12  }
```



Minimize Divergence Kernel

We have only changed these
two lines

```
01  __global__ void ConvergentSumReductionKernel(float* input, float* output) {
02      unsigned int i = threadIdx.x;
03      for (unsigned int stride = blockDim.x; stride >= 1; stride /= 2) {
04          if (threadIdx.x < stride) {
05              input[i] += input[i + stride];
06          }
07          __syncthreads();
08      }
09      if(threadIdx.x == 0) {
10          *output = input[0];
11      }
12  }
```



Minimize Divergence Kernel

How can we improve this within block kernel further?

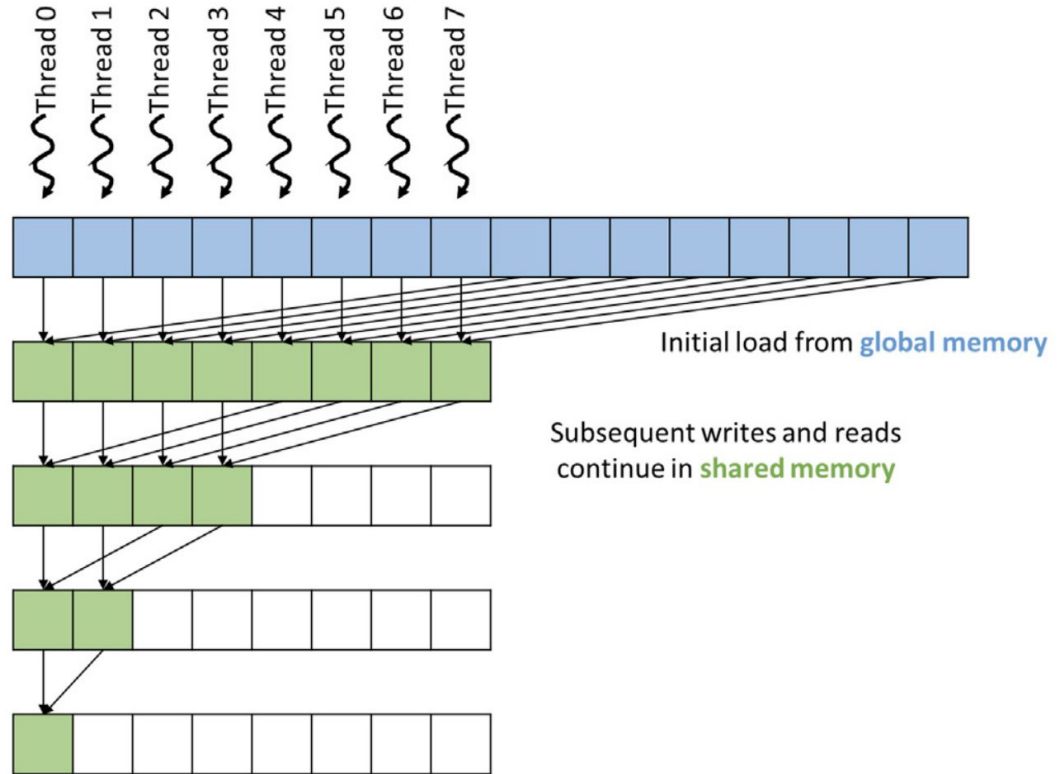
We have only changed these two lines

```
01  __global__ void ConvergentSumReductionKernel(float* input, float* output) {
02      unsigned int i = threadIdx.x;
03      for (unsigned int stride = blockDim.x; stride >= 1; stride /= 2) {
04          if (threadIdx.x < stride) {
05              input[i] += input[i + stride];
06          }
07          __syncthreads();
08      }
09      if(threadIdx.x == 0) {
10          *output = input[0];
11      }
12  }
```



Shared Memory Kernel

Repeat the same access pattern as in the last kernel, but only the first read and last write are from/to global memory.

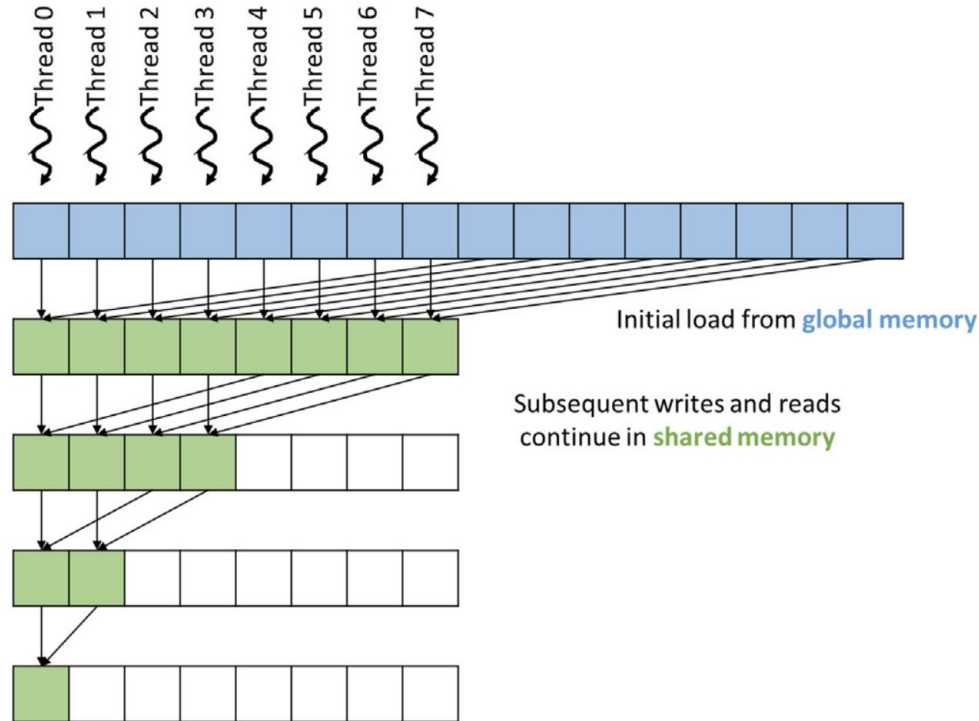


Shared Memory Kernel

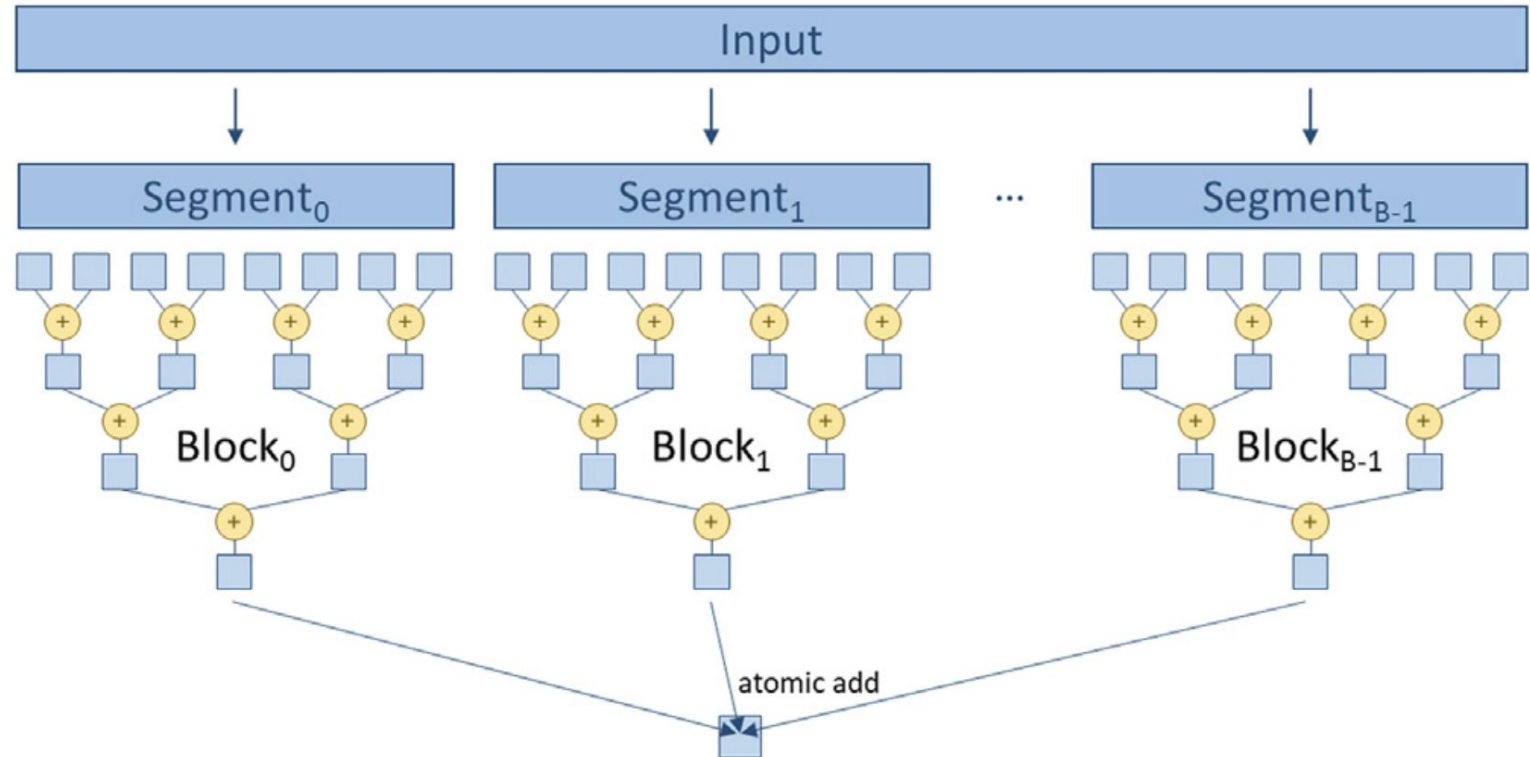
```
01  __global__ void SharedMemorySumReductionKernel(float* input) {
02      __shared__ float input_s[BLOCK_DIM];
03      unsigned int t = threadIdx.x;
04      input_s[t] = input[t] + input[t + BLOCK_DIM];
05      for (unsigned int stride = blockDim.x/2; stride >= 1; stride /= 2) {
06          __syncthreads();
07          if (threadIdx.x < stride) {
08              input_s[t] += input_s[t + stride];
09          }
10      }
11      if (threadIdx.x == 0) {
12          *output = input_s[0];
13      }
14  }
```



How can we extend this pattern to multiple blocks when the sequence length is greater than 2048?



A Hierarchical Reduction Kernel

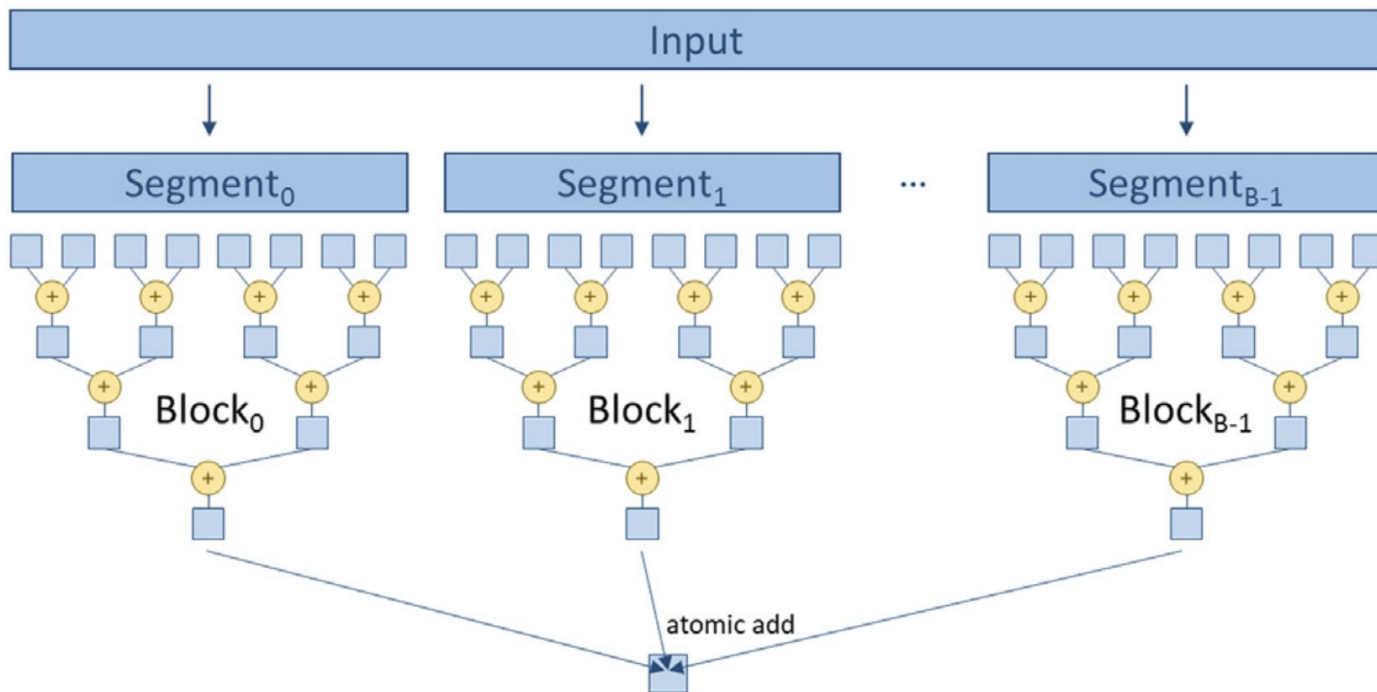


A Hierarchical Reduction Kernel

```
01  __global__ SegmentedSumReductionKernel(float* input, float* output) {
02      __shared__ float input_s[BLOCK_DIM];
03      unsigned int segment = 2*blockDim.x*blockIdx.x;
04      unsigned int i = segment + threadIdx.x;
05      unsigned int t = threadIdx.x;
06      input_s[t] = input[i] + input[i + BLOCK_DIM];
07      for (unsigned int stride = blockDim.x/2; stride >= 1; stride /= 2){
08          __syncthreads();
09          if (t < stride) {
10              input_s[t] += input_s[t + stride];
11          }
12      }
13      if (t == 0) {
14          atomicAdd(output, input_s[0]);
15      }
16  }
```

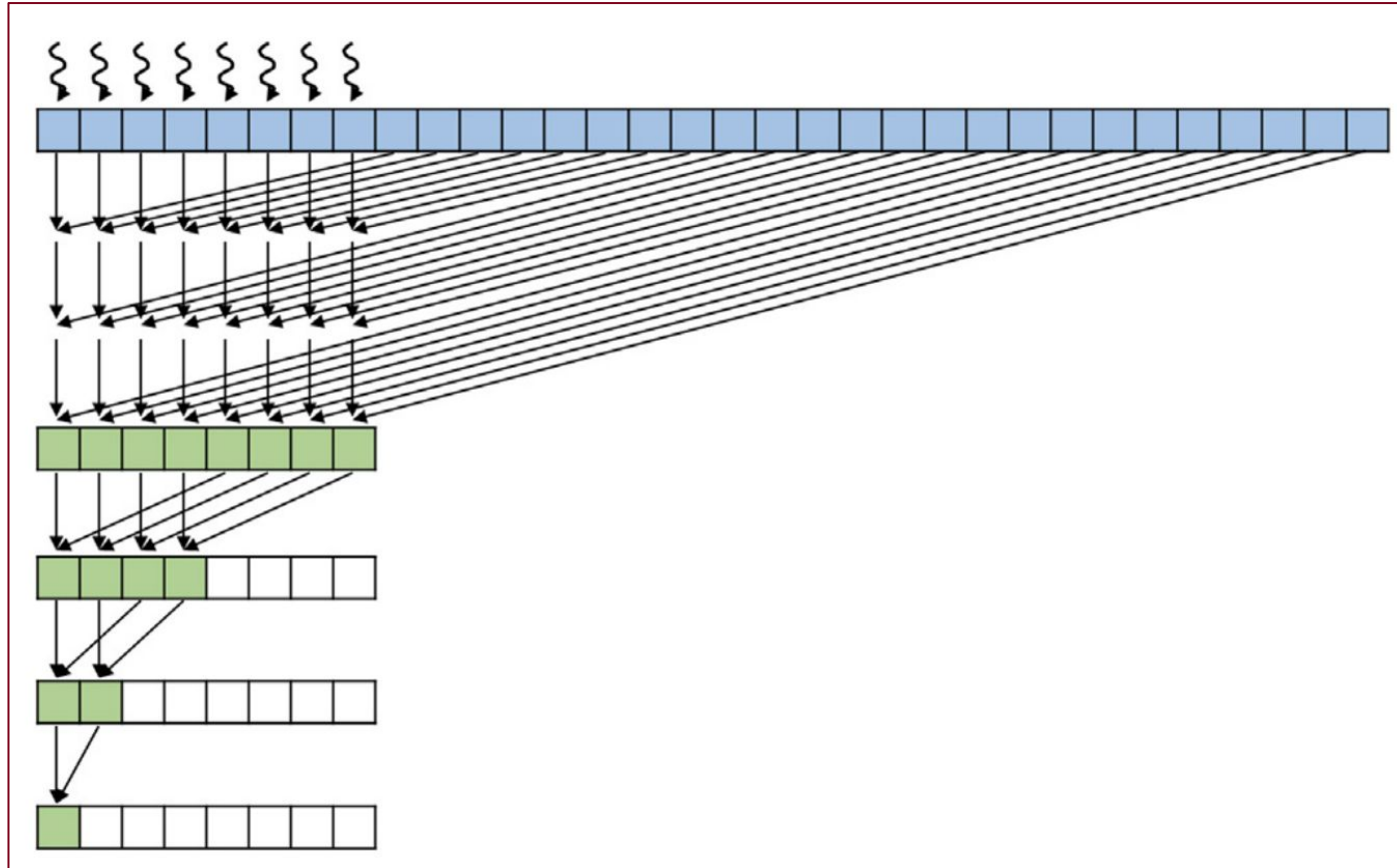


What further improvements can we make to this access pattern?

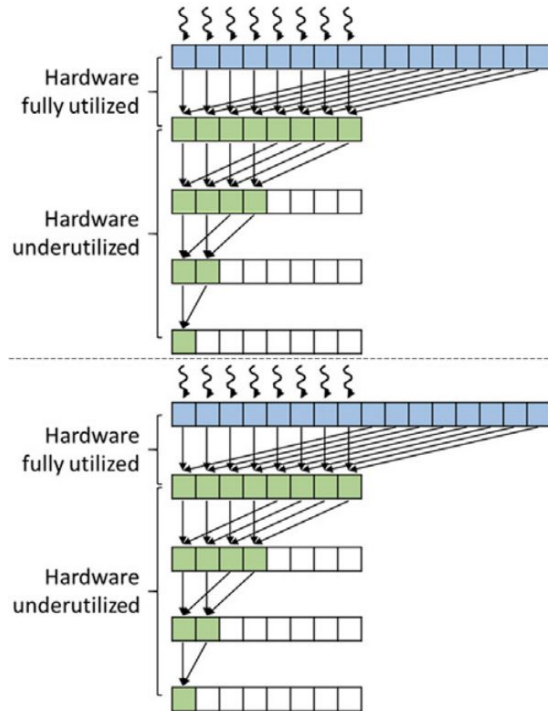


Coarsened Kernel

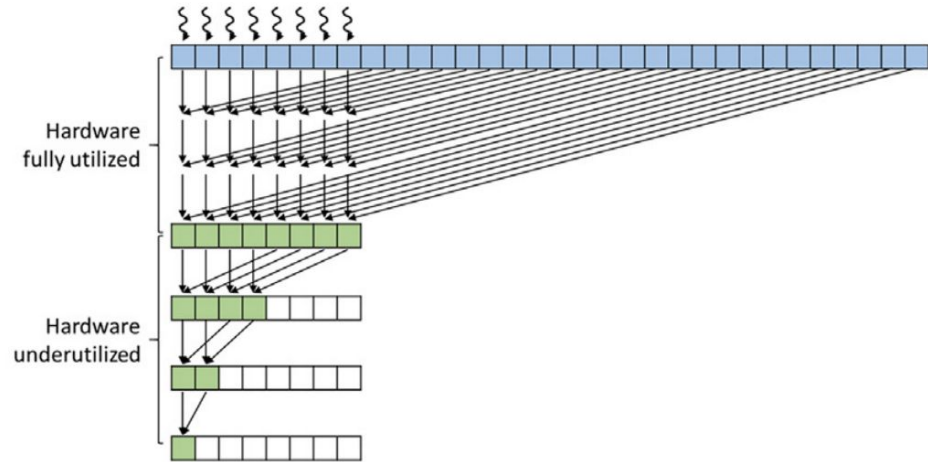
Each thread
computes
multiple
outputs



Coarsened Kernel



(A) Execution of two original thread blocks serialized by the hardware



(B) Execution of one coarsened thread block doing the work of two original thread blocks

Coarsened Kernel

```
01  __global__ CoarsenedSumReductionKernel(float* input, float* output) {
02      __shared__ float input_s[BLOCK_DIM];
03      unsigned int segment = COARSE_FACTOR*2*blockDim.x*blockIdx.x;
04      unsigned int i = segment + threadIdx.x;
05      unsigned int t = threadIdx.x;
06      float sum = input[i];
07      for(unsigned int tile = 1; tile < COARSE_FACTOR*2; ++tile) {
08          sum += input[i + tile*BLOCK_DIM];
09      }
10      input_s[t] = sum;
11      for (unsigned int stride = blockDim.x/2; stride >= 1; stride /= 2){
12          __syncthreads();
13          if (t < stride) {
14              input_s[t] += input_s[t + stride];
15          }
16      }
17      if (t == 0) {
18          atomicAdd(output, input_s[0]);
19      }
20  }
```

